

NOVA BANK

Full-Stack Digital Banking Web Application

Project Technical Report

Team FalconX

Secure. Smart. Yours.

Category	Details
Project Type	Full-stack Digital Banking Web Application
Framework	Next.js 16 (App Router) + React 19
Language	TypeScript 5.9.x
Database	PostgreSQL 17
Containerisation	Docker + Docker Compose
CI/CD	GitHub Actions

1. Project Overview	3
2. Technology Stack	3
3. System Architecture	4
3.1 Architecture Diagram	4
3.2 Key Architectural Decisions	4
4. Database Schema.....	6
4.1 Notable Constraints	6
5. Security Implementation.....	7
6. OTP Flow	8
7. API Endpoints	9
7.1 Customer Routes (Session Required)	9
7.2 Admin Routes (role = admin, Server-Verified).....	9
8. Customer Features	10
9. Admin Panel.....	10
10. Email System	11
11. CI/CD Pipeline	11
12. Development Setup.....	11
13. Project Statistics.....	13

1. Project Overview

Nova Bank is a secure, production-grade internet banking platform built as a full-stack digital banking web application. It enables customers to manage accounts, perform transfers, pay bills, and view spending analytics — complemented by a comprehensive administrative back-office.

2. Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.2.9
UI Library	React	19.2.4
Language	TypeScript	5.9.x
Database	PostgreSQL	17 (Alpine)
DB Client	node-postgres (pg)	8.22.x
Password Hashing	bcryptjs	3.0.x
Email (OTP)	nodemailer	9.0.x
Linters/Formatter	Biome	2.4.16
Styling	Tailwind CSS v4 + CSS Modules	4.3.x
Runtime	Node.js	20 (Alpine)
Containerisation	Docker + Docker Compose	—
CI/CD	GitHub Actions	—

3. System Architecture

Nova Bank follows a monolithic Next.js application pattern with co-located API routes and no separate backend service. The architecture prioritises security, atomicity, and developer ergonomics.

3.1 Architecture Diagram

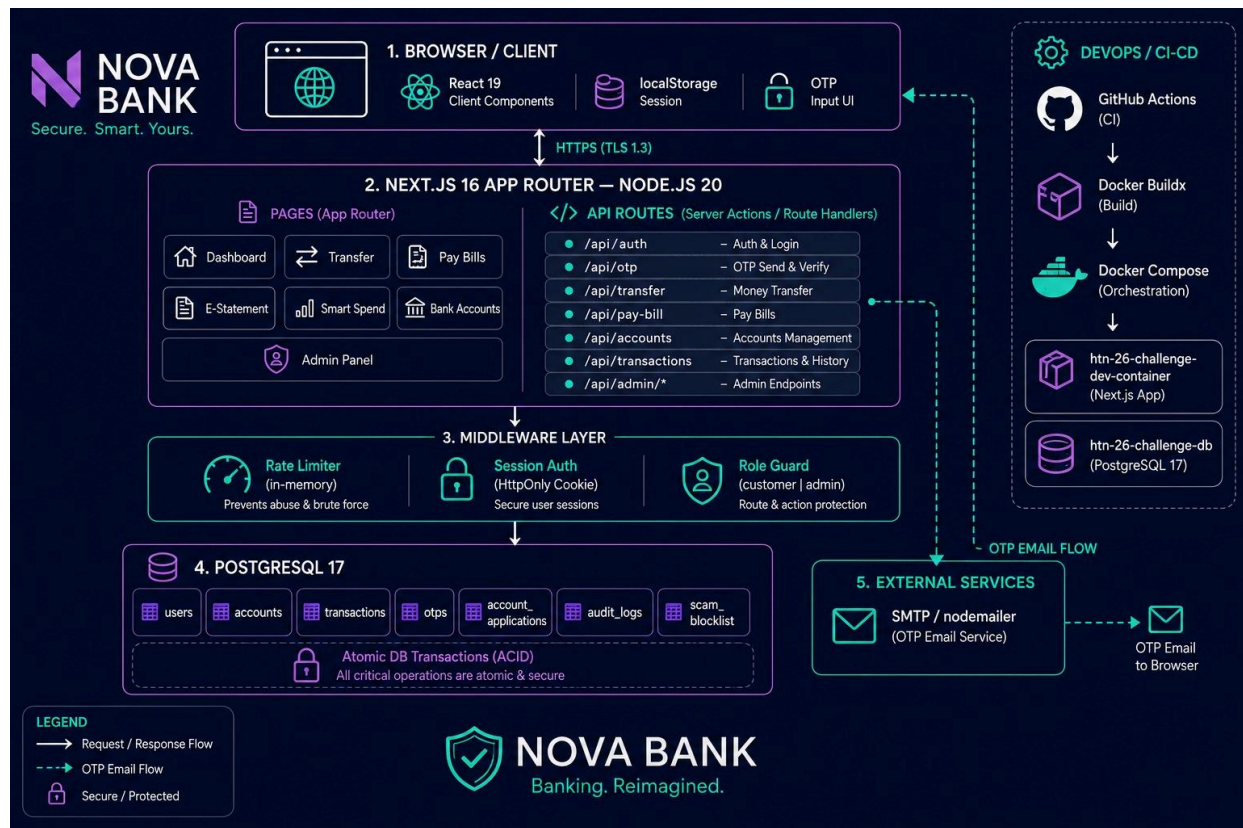


Figure 1: Nova Bank System Architecture Overview

3.2 Key Architectural Decisions

- No ORM — raw parameterised queries (\$1, \$2) throughout for full control and zero ORM-level abstraction vulnerabilities
- Idempotent schema migrations — CREATE TABLE IF NOT EXISTS + ALTER TABLE ... ADD COLUMN IF NOT EXISTS on every boot; no migration runner needed
- Atomic transactions — BEGIN / COMMIT / ROLLBACK with FOR UPDATE row locking on every financial operation, preventing race conditions under concurrent load
- Dual-session pattern — HttpOnly cookie (user_id, role) for server-side API auth; localStorage (nova_session) for client-side display data. Each layer trusts only its own source

- Virtual counterparties — BANK_SYSTEM and BILLER_SYSTEM string identifiers used as virtual accounts for admin deposits and bill payments
- serverExternalPackages — pg and nodemailer are marked as Node.js-only externals in next.config.ts, preventing webpack bundling into the browser

4. Database Schema

The application uses 8 PostgreSQL tables with strict relational integrity constraints and DB-level balance enforcement.

Table	Purpose
users	Customer and admin accounts (id, username, email, full_name, password_hash, role, created_at)
accounts	Bank accounts (id, user_id, account_number, account_type, balance, is_frozen, created_at)
transactions	All financial movements (id, from_account, to_account, amount, description, status, flagged)
otps	One-time passwords (id, user_id, code, purpose, expires_at, used, created_at)
account_applications	Account open requests (status, reject_reason, reviewed_by, reviewed_at)
audit_logs	Immutable event log (id, event, payload JSONB, created_at)
scam_blocklist	Flagged account numbers with reason and severity
risk_rules	Configurable fraud detection thresholds

4.1 Notable Constraints

- accounts.balance enforced ≥ 0 at database level
- otps.expires_at checked server-side before accepting any OTP
- Foreign keys: accounts.user_id $\rightarrow$ users.id, transactions.created_by $\rightarrow$ users.id, otps.user_id $\rightarrow$ users.id

5. Security Implementation

Nova Bank implements 14 distinct security controls covering all major threat vectors for a financial application.

Threat	Mitigation
SQL Injection	100% parameterised queries — zero string interpolation in SQL
Brute-force login	In-memory rate limiter: 5 attempts per 10 minutes per IP
OTP abuse	Rate limited: 5 OTP requests per 10 min per user+IP; single-use, 10-min expiry
Signup flooding	Rate limited: 5 accounts per hour per IP
PIN interception	PIN system removed — replaced with email OTP for every transfer and bill payment
Privilege escalation	Every /api/admin/* route verifies role = 'admin' from the HttpOnly session cookie server-side
Unauthorised data access	Customer API routes verify user_id ownership of every account
Race conditions	SELECT ... FOR UPDATE row-level lock on source account balance before debit
Negative balance	Balance sufficiency checked inside the locked transaction — rollback if insufficient
Frozen account transfers	Server-side check on both source and destination is_frozen flag
Reversal overdraft	Admin reversal verifies recipient has sufficient balance before debiting back
Password storage	bcryptjs with cost factor 10
Session fixation	Sessions stored server-side as HttpOnly cookies; client cannot read or forge role
Unauthenticated API calls	All API routes return 401 Unauthorized if no valid session cookie

6. OTP Flow

The OTP flow governs all sensitive financial operations including transfers and bill payments. The OTP is marked as used inside the same database transaction as the transfer — a crash between the two operations is impossible.

Step	Action
1	User fills form and clicks Continue
2	POST /api/otp { purpose: 'transfer' } — rate limit check (5/10 min)
3	Invalidate previous unused OTPs for same user + purpose
4	Generate 6-digit code, store in otps table (10 min expiry)
5	Send branded HTML email via nodemailer — return masked email address
6	User enters OTP and clicks Confirm
7	POST /api/transfer — verify OTP exists, not used, not expired
8	BEGIN transaction → SELECT balance FOR UPDATE (row lock)
9	Check balance >= amount → Debit source, credit destination
10	INSERT transaction record + audit log → UPDATE otp SET used = true → COMMIT

7. API Endpoints

7.1 Customer Routes (Session Required)

Method	Endpoint	Purpose
POST	/api/auth/login	Login, set HttpOnly cookie
POST	/api/auth/signup	Register new user
POST	/api/auth/logout	Clear session
GET	/api/accounts	List own accounts
GET	/api/transactions	Transaction history (account param required)
POST	/api/otp	Generate and email OTP
POST	/api/transfer	Execute transfer (OTP required)
POST	/api/pay-bill	Pay a bill (OTP required)
GET/POST	/api/account-applications	View / submit account open requests

7.2 Admin Routes (role = admin, Server-Verified)

Method	Endpoint	Purpose
GET/POST	/api/admin/applications	Review account applications
GET	/api/admin/customers	Customer list and account details
POST	/api/admin/customers	Freeze / unfreeze accounts
POST	/api/admin/deposit	Teller deposit or withdrawal
GET/POST	/api/admin/transactions	Monitor, flag, unflag, reverse
GET/POST	/api/admin/fraud	Scam blocklist CRUD
GET	/api/admin/system	System metrics
GET	/api/search	Global search (admin only)

8. Customer Features

Feature	Description
Dashboard	Greeting, total portfolio balance, account switcher, quick actions, live recent transactions
Bank Transfer	Multi-step: form → OTP verification → result. No PIN, no scam modal
Pay Bills	Biller grid (12 billers) → form → OTP → result. Bill payments recorded against BILLER_SYSTEM virtual account
E-Statement	Account selector, full transaction table with running balance, CSV download, print
Smart Spend Analytics	Categorised spending (Bills, Transfers, Other), bar chart percentages, rule-based AI insights
Bank Accounts	View all accounts with type and frozen status, submit account open requests, track application status

9. Admin Panel

Page	Capability
Dashboard	6 live metric cards: total customers, accounts, transactions, total deposits, pending applications, flagged transactions
Customers	Full customer list, view all accounts per customer, freeze/unfreeze individual accounts
Applications	Approve or reject account open requests with reason, auto-creates account on approval
Teller Ops	Post deposits and withdrawals to any customer account, every operation audit-logged
Transactions	Full transaction monitor, flag suspicious, unflag, reverse (with balance check)
Fraud Console	Scam blocklist CRUD (add/remove flagged account numbers with severity), risk rule display

10. Email System

- Transport: nodemailer over SMTP (Gmail, Brevo, or any provider)
- Dev fallback: If SMTP_HOST is not configured, the OTP is printed to the server console in a clearly formatted block — zero config needed for local development
- Template: Branded HTML email with Nova Bank gradient header (#1d0730 → #450043), large OTP display, amber warning box
- Supported purposes: transfer | payment — email subject and body language adapts accordingly

11. CI/CD Pipeline

Three sequential GitHub Actions jobs are triggered on push to main and pull requests targeting main.

Job	Steps	Fails On
code-quality	npm ci → tsc --noEmit → biome check	Any TypeScript error or lint/format violation
build	npm ci → next build	Bundle/compilation errors
docker	docker buildx build (no push)	Docker image build failure

- Docker layer caching via GitHub Actions cache (type=gha)
- No database service needed in CI — pg Pool is lazy, no connection at build time

12. Development Setup

The full stack runs with a single command:

```
docker compose up --build --watch
```

Environment Variable	Purpose
DATABASE_URL	PostgreSQL connection string
SMTP_HOST / PORT / USER / PASS / FROM	Email provider credentials
ADMIN_USERNAME / PASSWORD / EMAIL / FULL_NAME	Seeded admin account

- App: http://localhost:3000
- Database: PostgreSQL on port 5432

- Hot reload via Docker Compose Watch (sync action)
- Schema auto-created on first boot via ensureDatabase()
- .next cache wiped on every container start — prevents stale chunk errors

13. Project Statistics

Metric	Count
API Endpoints	18
Database Tables	8
Customer Pages	6
Admin Pages	6
Lines of Application Code	~4,000+
Docker Images	2 (dev + prod)
Security Controls Implemented	14

Nova Bank — Banking. Reimagined.